

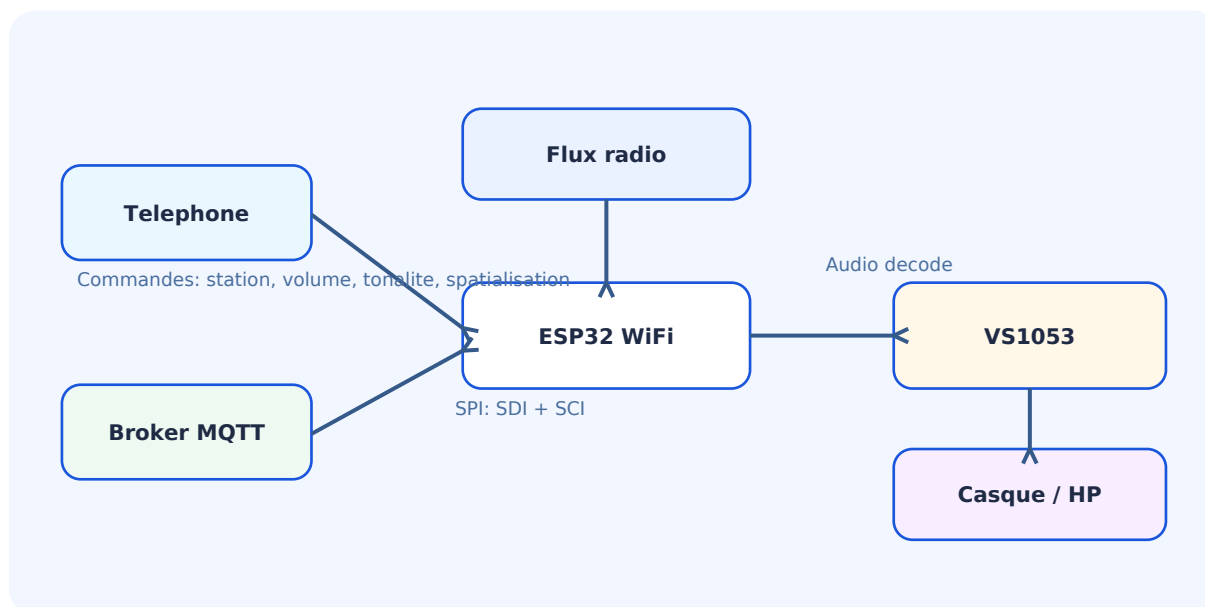
SAE 2.02 - Projet Web Radio portable

ESP32 - VS1053 - Arduino - WiFi - MQTT

Étudiant	Quentin ALTINOK
Formation	BUT 1 Réseaux & Télécommunications - IUT de Colmar
Semestre	Semestre 2
Durée	22 heures au total : 10 h encadrées et 12 h en autonomie
Compétences	C1 - Administrer les réseaux et l'Internet ; C3 - Créer des outils et applications informatiques pour les R&T;

Résumé

L'objectif de cette SAE était de réaliser une Web Radio portable capable de lire des stations de radio Internet codées en MP3 ou AAC. Le système devait être utilisable au casque ou sur haut-parleurs et permettre le contrôle du choix de la station, du volume, de la tonalité et de l'effet de spatialisation depuis un téléphone. La solution repose sur une carte Adafruit HUZZAH32 ESP32 Feather, une carte audio Music Maker FeatherWing intégrant un codec VS1053, l'environnement Arduino IDE, une connexion WiFi et le protocole MQTT.



1. Contexte et objectifs

Dans ce projet, il fallait associer un microcontrôleur connecté à Internet avec un circuit spécialisé dans le décodage audio. La carte ESP32 récupère un flux radio en ligne, communique avec le codec VS1053 et permet de transmettre les commandes reçues depuis un téléphone. Le projet demandait donc à la fois une compréhension matérielle, une adaptation logicielle et une logique de communication réseau.

- écouter des radios Internet en stéréo via des flux MP3 ou AAC ;
- utiliser l'ESP32 pour la connexion WiFi et le traitement des commandes ;
- communiquer avec le VS1053 en SPI pour décoder le flux audio ;
- gérer le volume, la tonalité, la spatialisation et les URL de stations ;
- piloter la Web Radio depuis un téléphone à l'aide de MQTT.

Le sujet prévoyait une progression par validations : premier programme de Web Radio, amélioration du son, gestion du WiFi, gestion des URL, ajout du MQTT, puis synchronisation des informations avec l'interface mobile.

2. Matériel et logiciels utilisés

Élément	Rôle dans le projet
Adafruit HUZZAH32 ESP32 Feather	Carte principale du projet. Elle fournit le microcontrôleur, la connexion WiFi, les GPIO et l'exécution du programme Arduino.
Adafruit Music Maker FeatherWing	Carte audio complémentaire contenant le VS1053 et les connexions nécessaires pour la sortie audio.
Codec VS1053	Circuit de décodage audio utilisé pour lire les flux MP3/AAC et gérer certains traitements sonores comme le volume, les basses, les aigus et la spatialisation.
Arduino IDE	Environnement de développement utilisé pour modifier, compiler et téléverser le programme sur l'ESP32.
Bibliothèques Arduino	ESP32 by Espressif, ESP_VS1053_Library / Baldram, WiFiManager, PubSubClient et bibliothèques nécessaires au WiFi, au flux audio et au MQTT.
MQTT / IoT MQTT Panel	Protocole et application mobile permettant de contrôler la radio à distance depuis un téléphone.

3. Architecture technique

L'architecture du système peut être séparée en quatre parties. Le téléphone sert d'interface de commande. Le broker MQTT transporte les messages de contrôle. L'ESP32 assure la connexion WiFi, reçoit les commandes et récupère les flux radio. Le VS1053 reçoit les données et les commandes par SPI, puis décode le son vers la sortie casque ou haut-parleurs.

Le VS1053 possède deux interfaces logiques principales sur le bus SPI :

- SDI - Serial Data Interface : utilisée pour envoyer les données audio compressées au codec.
- SCI - Serial Control Interface : utilisée pour écrire ou lire les registres de contrôle du codec, par exemple pour agir sur le volume, la tonalité ou certains modes de traitement.

Le registre SCI_MODE a notamment été utilisé dans la logique de spatialisation : le programme lit le registre, modifie certains bits selon le mode choisi, puis réécrit la nouvelle valeur dans le VS1053. Cette partie montre l'intérêt de comprendre le fonctionnement bas niveau du codec et pas uniquement d'utiliser une bibliothèque.

4. Déroulement du travail

- 1 Étude du sujet et identification des fonctions attendues : lecture de Web Radio, réglage audio, WiFi, URL, MQTT.
- 2 Découverte du matériel : ESP32, Music Maker FeatherWing, VS1053, broches GPIO et brochage SPI.
- 3 Configuration de l'IDE Arduino, installation de la carte ESP32 et des bibliothèques nécessaires.
- 4 Test d'un premier exemple de Web Radio et adaptation du programme aux cartes utilisées.
- 5 Ajout des réglages de volume, de tonalité et de spatialisation du son.
- 6 Amélioration de la connexion WiFi avec WiFiManager pour éviter de figer les identifiants dans le code.
- 7 Gestion des stations et des URL de flux radio, avec possibilité de choisir une station personnalisée.
- 8 Ajout du protocole MQTT pour contrôler le lecteur depuis un téléphone.
- 9 Tests, corrections, validation du fonctionnement et rédaction de la trace portfolio.

5. Compétences mobilisées

Apprentissage critique	Justification dans la SAE
AC11.02	Compréhension d'un système connecté : flux audio Internet, WiFi, ESP32, bus SPI et codec audio.
AC11.05	Identification des dysfonctionnements : erreurs de connexion WiFi, broker MQTT non joignable, flux radio invalide, problème de téléversement ou de communication SPI.
AC13.01	Utilisation d'outils informatiques : Arduino IDE, moniteur série, bibliothèques, compilation et téléversement sur microcontrôleur.
AC13.02	Lecture, correction et modification d'un programme existant pour ajouter des fonctions audio, WiFi, URL et MQTT.
AC13.04	Présentation du projet dans une page portfolio structurée en HTML/CSS avec illustrations et retour d'expérience.

6. Tests réalisés

Les tests ont été réalisés progressivement afin de vérifier chaque partie avant d'ajouter la suivante. La première vérification concernait la lecture d'une station Web Radio. Ensuite, chaque réglage audio a été testé séparément : volume, tonalité, basses, aigus et spatialisation. Les tests WiFi ont permis de confirmer la connexion de l'ESP32 au réseau. Les tests MQTT ont ensuite validé l'envoi de commandes depuis le téléphone et leur prise en compte dans le programme.

- lecture correcte d'un flux radio Internet ;
- modification du volume et de la tonalité audible à la sortie audio ;
- changement de station et ajout d'une URL personnalisée ;
- pilotage depuis l'application mobile via MQTT ;
- corrections grâce aux messages du moniteur série Arduino.

7. Difficultés rencontrées et solutions

Difficulté	Solution ou méthode utilisée
Téléversement vers l'ESP32	Vérification de la carte sélectionnée, du port COM et de l'installation du support ESP32 dans l'IDE Arduino.
Compréhension du SPI	Repérage des broches utiles et distinction entre SDI pour les données audio et SCI pour les commandes du codec.
Flux radio instable ou invalide	Test de plusieurs URL, choix de flux compatibles MP3/AAC et affichage d'informations dans le moniteur série.
Commandes MQTT	Organisation des topics et test progressif des commandes : volume, station, URL et spatialisation.

8. Bilan personnel

Cette SAE m'a permis de progresser dans la compréhension des systèmes embarqués connectés. J'ai mieux compris le rôle de l'ESP32, l'intérêt du bus SPI et le fonctionnement général d'un codec audio externe. J'ai également appris à modifier un programme existant pour ajouter des fonctionnalités concrètes. Le pilotage par MQTT m'a permis de faire le lien entre un objet connecté, un réseau WiFi et une interface mobile.

Avec du recul, j'organiserais davantage le code dès le début en séparant les parties WiFi, MQTT, lecture audio et réglages sonores. Je documenterais aussi plus précisément les topics MQTT et les valeurs attendues afin de faciliter les tests et la maintenance.

9. Sources utilisées

- Sujet SAE 2.02 - Projet de Web Radio - ESP32 et VS1053.
- Datasheet VS1053b - VLSI Solution, version 1.32.
- Documentation Arduino IDE, ESP32, Adafruit HUZZAH32 ESP32 Feather et Music Maker FeatherWing.
- Bibliothèque ESP_VS1053_Library / Baldram et application IoT MQTT Panel.